Módulo 04: Diagnose and Fix a Failing Deployment in Azure Container Apps

Ciclo de Vida de Contenedores Serverless, Diagnóstico de Revisiones, Port Mismatch, Variables de Entorno y Observabilidad con Log Analytics

Alineación Curricular Oficial Microsoft Learn & Examen AI-200

CURSO OFICIAL AI-200T00-A

- **Curso Oficial:** AI-200T00-A: Developing Azure AI Solutions in the Cloud (Módulo 02 de Contenedores / Diapositivas 15 a 24).
- **Certificación Evaluada:** Microsoft Certified: Azure AI Cloud Developer Associate (Examen AI-200).
- **Dominio Técnico:** *Dominio 1: Desarrollo de soluciones en contenedores en Azure (Ponderación oficial: 20% – 25%).*
- **Guía de Estudio Oficial:** https://learn.microsoft.com/es-es/credentials/certifications/resources/study-guides/ai-200

PRERREQUISITOS UNIVERSALES DE EJECUCIÓN

- **Suscripción de Azure Activa:** Permisos RBAC de `Contributor`, `AcrPush` y `Monitoring Contributor`.
- **Azure CLI 64-bit (v2.50+):** Instalador nativo x64 (`C:\Program Files\Microsoft SDKs\Azure\CLI2`).
- **Entorno de Desarrollo:** Python 3.10+ de 64 bits y Visual Studio Code con terminal PowerShell o Bash.
- **Cero Daemon Local de Docker:** Compilación serverless en la nube delegada a Azure Container Registry Tasks.

CONSIDERACIONES CRÍTICAS Y NOVEDADES

- **Actualización de Modelo de IA:** Sustitución de `gpt-5.4-mini` por `gpt-5.1` y cuota `TPM_LIMIT = "5000"`.
- **Retrasos Transitorios de Azure:** Esperar estado `Succeeded` en el Environment para evitar fallos de despliegue.
- **Sintaxis Terminal Dual:** PowerShell utiliza acento grave (`` ` ``); Bash utiliza barra invertida (`\`).
- **Unicidad Global de Nombres:** Hashing criptográfico SHA-1 para garantizar FQDNs DNS globalmente únicos.

Estructura del Proyecto Starter (CODE/4. aca-manage-python)

mslearn-azure-ai / aca-manage-python

- `azdeploy.py`: Motor de despliegue automatizado, cálculo de hashes SHA-1, registro de extensiones y proveedores ARM.
- `api/Dockerfile`: Manifiesto OCI multicapa con imagen base `python:3.12-slim` y servidor WSGI Gunicorn en puerto 8000.
- `api/main.py`: Microservicio REST Flask con endpoints `/`, `/health`, `/process` y configuración Twelve-Factor.
- `api/requirements.txt`: Dependencias fijadas (`flask`, `gunicorn`, `pydantic`) compiladas con `--no-cache-dir`.

Anatomía Didáctica de Comandos Azure CLI Utilizados en el Laboratorio

Elemento / Flag CLI	Sintaxis de Ejemplo Aplicada	Propósito Arquitectónico y Rationale
`az containerapp update`	`--set-env-vars MODEL_NAME="gpt-5.1"`	Actualiza variables de entorno del contenedor, generando una **nueva revisión inmutable** activa.
`--remove-env-vars`	`--remove-env-vars MODEL_NAME`	Elimina claves de entorno para inducir o diagnosticar fallos de configuración Twelve-Factor.
`az containerapp ingress update`	`--target-port 8000`	Alinea el proxy perimetral Envoy con el socket TCP interno donde escucha el servidor de aplicaciones WSGI.
`az containerapp logs show`	`-n $APP_NAME -g $RG --follow`	Transmite en tiempo real los registros recientes de consola (stdout/stderr) almacenados en memoria volátil.
`az monitor log-analytics query`	`-w $WORKSPACE_ID --analytics-query ...`	Ejecuta consultas KQL sobre el repositorio centralizado persistente para auditoría y diagnóstico forense.

Tarea 0 & Fase 1: Instalación de Azure CLI 64-bit y Registro de Resource Providers

PREPARACIÓN BASE

El laboratorio requiere la versión de 64 bits de Azure CLI para evitar desbordamientos de memoria en consultas JSON complejas. Asimismo, deben registrarse los proveedores ARM y agregar las extensiones oficiales antes de iniciar el despliegue.

```
# 1. Comprobar arquitectura de 64 bits de Azure CLI
az version

# 2. Agregar extensiones oficiales para Azure Container Apps y Log Analytics
az extension add --name containerapp --upgrade
az extension add --name log-analytics --upgrade

# 3. Registrar Resource Providers esenciales en la suscripción de Azure
az provider register --namespace Microsoft.App
az provider register --namespace Microsoft.OperationalInsights
az provider register --namespace Microsoft.ContainerRegistry
```

Navegación Visual en Azure Portal: Diríjase a `Suscripciones > [Su Suscripción] > Proveedores de recursos`. Verifique que `Microsoft.App`, `Microsoft.OperationalInsights` y `Microsoft.ContainerRegistry` figuren en estado **Registered**.

Desglose Técnico del Código del Laboratorio: azdeploy.py, Dockerfile y main.py

Análisis Exhaustivo de la Lógica de Aprovisionamiento, Empaquetado OCI y Microservicio Flask

1. AZDEPLOY.PY — MOTOR DE AUTOMATIZACIÓN Y RESILIENCIA EN AZURE CLI

El script `azdeploy.py` implementa un orquestador automatizado en Python que interactúa directamente con la API de Azure CLI mediante `subprocess`. Sus responsabilidades de diseño clave incluyen:

- **Unicidad Global mediante Hashing Criptográfico SHA-1:**
  Calcula un hash de 13 caracteres basado en el nombre del grupo de recursos y la suscripción (`hashlib.sha1(...)[:13]`) para garantizar nombres de ACR y dominios de Container Apps globalmente únicos sin colisiones DNS.
- **Comprobación Idempotente y Detección de Proveedores:**
  Verifica e instala dinámicamente las extensiones `containerapp` y `log-analytics`, registrando los namespaces `Microsoft.App`, `Microsoft.OperationalInsights` y `Microsoft.ContainerRegistry` si no están activos.
- **Menú de Despliegue Modular en 5 Fases:**
  Opción 1: Compilación serverless con ACR Tasks; Opción 2: Creación del ACA Environment con bucle de reintento; Opción 3: Despliegue de Container App e inyección de secretos; Opción 5: Salida del script.
- **Manejo de Retrasos Transitorios (Transient Delays):**
  Monitorea activamente `properties.provisioningState == "Succeeded"` en el Managed Environment para mitigar el error bloqueante `ManagedEnvironmentNotProvisioned` en la fase subsiguiente.
- **Generación de Entorno Multiplataforma:**
  Exporta los valores resueltos a `.env.ps1` (PowerShell) y `.env.sh` (Bash/macOS), incluyendo `CONTAINER_APP_NAME`, `ACA_ENVIRONMENT`, `ACR_NAME`, `RESOURCE_GROUP` y `TPM_LIMIT`.

2. API/DOCKERFILE — EMPAQUETADO OCI DE PRODUCCIÓN Y CACHÉ DE CAPAS

El contenedor está diseñado siguiendo las directrices de empaquetado OCI seguro y eficiente para microservicios de IA:

- **Imagen Base Minimalista (`python:3.12-slim`):**
  Reduce el tamaño final de la imagen a menos de 150 MB y mitiga vulnerabilidades CVE al prescindir de utilidades innecesarias del sistema operativo Debian.
- **Higiene y Optimización de Capas de Caché:**
  Copia primero `requirements.txt` y ejecuta `pip install --no-cache-dir` antes de copiar `main.py`, garantizando que cambios frecuentes en el código de la API no invaliden la compilación de paquetes.
- **Configuración Twelve-Factor y Declaración de Puertos:**
  Define `ENV PORT=8000` y expone `EXPOSE 8000`, sirviendo como metadato documental para el balanceador perimetral de Container Apps.
- **Servidor WSGI de Alto Rendimiento (Gunicorn):**
  Ejecuta `gunicorn --bind 0.0.0.0:${PORT:-8000} --workers 2 --access-logfile - main:app`, enviando todos los accesos y errores a `stdout` para su captura automática por Azure Monitor.

3. API/MAIN.PY — MICROSERVICIO REST FLASK Y DIAGNÓSTICO DE SALUD

Microservicio web estructurado para tolerar fallos transitorios e integrarse con sondas de orquestación:

- **Patrón de Fallback Dinámico en Variables de Entorno:**
  `MODEL_NAME = os.getenv("MODEL_NAME", "not-configured")`: Permite que el microservicio inicie incluso si la variable de entorno no se inyectó, permitiendo diagnosticar la anomalía sin colapsar el pod.
- **Sondeo de Salud Desacoplado (`GET /health`):**
  Retorna `{"status":"healthy"}` con código HTTP 200 de forma inmediata sin depender de servicios externos de red, satisfaciendo los requisitos de Liveness Probe de Container Apps.
- **Descubrimiento de Metadatos del Entorno (`GET /`):**
  Expone un JSON con la versión de la API, el entorno de ejecución (`ENVIRONMENT`) y el objeto `model` con su nombre asignado, permitiendo validar la configuración desde el cliente.
- **Trazabilidad Estructurada con UUIDs de Solicitud:**
  Las operaciones en `POST /process` generan un `request_id` único para correlacionar transacciones distribuidas en Log Analytics sin registrar datos confidenciales (prompts o PII).

Aprovisionamiento de Container Apps: Comparativa Visual Azure Portal (GUI) vs. Azure CLI

Paridad Completa entre Comandos de Terminal y Asistentes Visuales de Azure Portal

Principio de Paridad Operativa Nube: Toda infraestructura aprovisionada por scripts (`azdeploy.py`) o comandos CLI puede ser inspeccionada, creada y gobernada visualmente en Azure Portal a través de sus asistentes de configuración y hojas de gestión.

Aprovisionamiento de Container Apps Environment en Azure Portal

GUI WIZARD

Navegación: Azure Portal > Entornos de Container Apps > + Crear

[Datos básicos] [Redes] [Supervisión] [Etiquetas] [Revisar y crear]

Pestaña / Campo GUI	Valor / Selección en Laboratorio	Equivalente CLI y Rationale Arquitectónico
[Datos básicos] Nombre de entorno	`managedEnvironment-<hash>`	`az containerapp env create -n ...`. Límite de seguridad y aislamiento para contenedores.
[Datos básicos] Región	`East US` (o su región preferida)	`--location eastus`. Co-ubicación con el grupo de recursos para minimizar latencia de red.
[Supervisión] Log Analytics Workspace	Seleccionar área de trabajo existente	`--logs-workspace-id ...`. Vincula el motor KQL para persistencia de logs de consola y sistema.
[Redes] Red virtual (VNet)	Infraestructura administrada (predeterminada)	Sin `--infrastructure-subnet-id`. Azure gestiona la VNet y el proxy Envoy de forma transparente.

Despliegue y Configuración de Container App en Azure Portal

GUI WIZARD

Navegación: Azure Portal > Container Apps > + Crear

[Datos básicos] [Contenedor] [Entrada (Ingress)] [Supervisión] [Revisar y crear]

Pestaña / Campo GUI	Valor / Selección en Laboratorio	Equivalente CLI y Rationale Arquitectónico
[Contenedor] Origen de imagen	Azure Container Registry (ACR)	`--registry-server ...`. Habilita credenciales administradas o admin para pull de imagen.
[Contenedor] Imagen y etiqueta	`ai-api:latest` (o digest SHA-256)	`--image <acr>.azurecr.io/ai-api:latest`. Binario del microservicio compilado.
[Contenedor] Variables de entorno	`MODEL_NAME = gpt-5.1`, `PORT = 8000`	`--env-vars MODEL_NAME=...`. Inyección Twelve-Factor en el runtime del contenedor.
[Entrada] Habilitada & Tráfico	Habilitada / Aceptar tráfico desde cualquier lugar	`--ingress external`. Expone el microservicio a Internet mediante FQDN público con TLS.
[Entrada] Puerto de destino (Target Port)	`8000` (Coincide con Gunicorn)	`--target-port 8000`. El proxy Envoy redirige peticiones HTTPS entrantes al socket 8000.

CUCHILLAS DE INSPECCIÓN POST-DESPLIEGUE EN AZURE PORTAL

- **Información general (Overview):** Verifique la *URL de la aplicación* (FQDN HTTPS), el estado del aprovisionamiento (*Succeeded*) y las réplicas activas (1/1).
- **Administración de revisiones (Revision management):** Muestra el historial inmutable de revisiones creadas, el porcentaje de tráfico (100%) y el estado operativo de cada réplica.
- **Entrada (Ingress):** Permite modificar el `Target Port`, tipo de transporte (Auto/HTTP1/HTTP2) y restricciones IP sin reconstruir la imagen.
- **Configuración (Configuration):** Permite agregar, editar o remover variables de entorno y secretos; al guardar, Azure Container Apps **gatilla automáticamente una nueva revisión**.
- **Transmisión de registros (Log stream):** Visualización en tiempo real de stdout/stderr de la aplicación, esencial para depuración inmediata.

Fase 1: Despliegue de Servicios Base con azdeploy.py y Verificación de Ingress

Compilación Serverless en ACR Tasks, Despliegue de Container App y Carga de Variables

Paso 1: Configuración de Parámetros y Modelo gpt-5.1 en azdeploy.py

CONFIGURACIÓN

Abra `azdeploy.py` en VS Code y configure sus valores de infraestructura. Realice un Find & Replace global para actualizar el modelo a `gpt-5.1` e incorpore la variable `TPM_LIMIT`:

```
# Modificar valores al inicio de azdeploy.py
rg = "rg-ai200-containers"     # Ingrese su grupo de recursos
location = "eastus"            # Región Azure (ej. eastus)

# Reemplazo global: Buscar 'gpt-5.4-mini' y reemplazar por 'gpt-5.1'
MODEL_NAME = "gpt-5.1"
TPM_LIMIT = "5000"             # Agregar tras MODEL_NAME

# En la función write_env_files(), asegurar la inclusión de TPM_LIMIT:
"TPM_LIMIT": TPM_LIMIT,
```

Opciones 1 y 2 en azdeploy.py

ACR & ENV

Ejecute el script y seleccione secuencialmente las opciones de infraestructura base:

```
python azdeploy.py

# Seleccionar '1': Compilación ACR Tasks
# Genera tarball de api/ y compila ai-api:latest

# Seleccionar '2': Crear ACA Environment
# IMPORTANTE: Si reporta timeout de conexión,
# verifique el estado hasta que retorne 'Succeeded':
az containerapp env show `
  -n $env:ACA_ENVIRONMENT -g $env:RESOURCE_GROUP `
  --query properties.provisioningState -o tsv
```

Opción 3 y Carga de .env.ps1

APP DEPLOY

Despliegue la Container App e importe las variables a su sesión de terminal:

```
# Seleccionar '3': Desplegar Container App
# Inyecta secretos, variables y target-port 8000

# Seleccionar '5': Salir del script

# Cargar variables en la sesión activa
. .\.env.ps1     # En PowerShell
# source .env.sh # En Bash / macOS

# Obtener el FQDN público del Ingress
$FQDN = az containerapp show `
  -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP `
  --query properties.configuration.ingress.fqdn -o tsv
```

Verificación Basal de la API y Respuesta del Modelo

VERIFICACIÓN HTTP

Compruebe que el microservicio esté en ejecución y responda con los metadatos correctos del modelo `gpt-5.1`:

```
# Consultar el endpoint raíz del microservicio desplegado
Invoke-RestMethod -Uri "https://$FQDN/"

# Salida JSON esperada de verificación:
{
  "status": "running",
  "environment": "production",
  "model": {
    "name": "gpt-5.1",
    "tpm_limit": "5000"
  }
}
```

Validación en Azure Portal: Navegue a `Container Apps > ai-api > Información general`. Copie la *URL de la aplicación* en el navegador para verificar la respuesta JSON directamente desde la web.

Fase 2: Diagnóstico y Resolución de Variables de Entorno Faltantes

Degradación Silenciosa de Microservicios, Inmutabilidad de Revisiones y Restauración de Configuración

Problemática Arquitectónica: Cuando una variable de configuración crítica (Twelve-Factor) se omite o elimina, microservicios resilientes pueden no colapsar gracias a valores por defecto, pero sufren una **degradación silenciosa** que afecta a los clientes downstream sin generar excepciones fatales inmediatas.

Paso 1 & 2: Eliminación de MODEL_NAME y Creación Automática de Revisión

ANOMALÍA INDUCIDA

Elimine la variable MODEL_NAME del contenedor y observe cómo Container Apps crea de inmediato una nueva revisión inmutable:

```
# 1. Eliminar la variable MODEL_NAME del contenedor
az containerapp update `
  -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP `
  --remove-env-vars MODEL_NAME

# 2. Listar las revisiones para confirmar la creación de una nueva versión inmutable
az containerapp revision list `
  -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP -o table
```

Comportamiento del Modelo de Revisiones: Cada mutación en la sección `properties.template` (variables, imagen, CPU o réplicas) genera una **nueva instantánea inmutable (Revisión)**. Por defecto, en modo *Single Revision*, el 100% del tráfico se transfiere inmediatamente a la nueva revisión.

Paso 3: Detección del Síntoma

PERSPECTIVA CLIENTE

Consulte la API para constatar la degradación del modelo:

```
(Invoke-RestMethod -Uri "https://$FQDN/").model

# Salida observada:
name          tpm_limit
----          ---------
not-configured 5000
```

El valor `not-configured` revela que `os.getenv("MODEL_NAME")` retornó su fallback.

Paso 4: Inspección de Configuración

CAUSA RAÍZ

Inspeccione las variables de entorno inyectadas en el pod:

```
az containerapp show `
  -n $env:CONTAINER_APP_NAME `
  -g $env:RESOURCE_GROUP `
  --query "properties.template.containers[0].env" `
  -o table
```

La tabla confirma que `MODEL_NAME` ya no existe en el contenedor.

Paso 5 & 6: Reincorporación de la Variable y Verificación de Restauración

RESOLUCIÓN EFECTIVA

Restaure la variable de entorno mediante `--set-env-vars` y verifique que la API responda nuevamente con `gpt-5.1`:

```
# 5. Reinyectar la variable MODEL_NAME (crea una 3ra revisión activa)
az containerapp update `
  -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP `
  --set-env-vars MODEL_NAME=$env:MODEL_NAME

# 6. Validar que la API retorne el nombre de modelo correcto
(Invoke-RestMethod -Uri "https://$FQDN/").model

# Salida esperada tras la corrección:
name    tpm_limit
----    ---------
gpt-5.1 5000
```

Navegación Visual en Azure Portal: Diríjase a `Container Apps > ai-api > Configuración > Variables de aplicación`. Observe cómo la variable `MODEL_NAME` se encuentra presente y, en la sección `Administración de revisiones`, compruebe que la última revisión creada gestiona ahora el 100% del tráfico de red.

Fase 3: Diagnóstico y Corrección de Port Mismatch e Ingress 503

Enrutamiento Envoy Ingress, Desajuste de Target Port, Análisis de Logs y Restauración

Problemática Arquitectónica: Azure Container Apps utiliza **Envoy como proxy inverso perimetral**. Si el parámetro `target-port` configurado en el Ingress difiere del puerto TCP en el que el servidor WSGI (Gunicorn) está vinculado (`0.0.0.0:8000`), Envoy no puede establecer conexión TCP con el pod, generando un error global **HTTP 503 Service Unavailable (Connection Refused)** en todas las rutas.

Paso 1 & 2: Inducción de Desajuste de Puerto (Target Port 3000) y Error 503

PORT MISMATCH

Modifique el puerto de destino del Ingress a 3000 e intente consumir el endpoint de sondeo de salud:

```
# 1. Configurar el puerto de entrada a 3000 (desajuste con Gunicorn en 8000)
az containerapp ingress update `
  -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP `
  --target-port 3000

# 2. Intentar consultar el endpoint de salud desde el cliente HTTP
Invoke-RestMethod -Uri "https://$FQDN/health"

# Error HTTP arrojado en el cliente:
# Invoke-RestMethod: The remote server returned an error: (503) Server Unavailable.
# Detalle interno Envoy: "upstream connect error or disconnect/reset before headers. reset reason: connection failure"
```

Paso 3a: Inspección de Ingress

PLANO DE ENTRADA

Verifique la configuración activa en el proxy perimetral Envoy:

```
az containerapp show `
  -n $env:CONTAINER_APP_NAME `
  -g $env:RESOURCE_GROUP `
  --query "properties.configuration.ingress" `
  -o yaml

# Evidencia:
fqdn: ai-api.wonderfulplant-...
targetPort: 3000
```

Paso 3b: Logs del Contenedor

PLANO DE CÓMPUTO

Compruebe si la app está viva y en qué puerto escucha:

```
az containerapp logs show `
  -n $env:CONTAINER_APP_NAME `
  -g $env:RESOURCE_GROUP

# Evidencia en salida Gunicorn:
[INFO] Starting gunicorn 22.0.0
[INFO] Listening at: http://0.0.0.0:8000
```

Paso 4 & 5: Corrección de Target Port a 8000 y Verificación de Salud

RESOLUCIÓN EFECTIVA

Alinee el puerto de destino del Ingress con el puerto 8000 y confirme la recuperación de la API:

```
# 4. Restablecer el target-port correcto a 8000
az containerapp ingress update `
  -n $env:CONTAINER_APP_NAME -g $env:RESOURCE_GROUP `
  --target-port 8000

# 5. Verificar la accesibilidad del endpoint de sondeo de salud
Invoke-RestMethod -Uri "https://$FQDN/health"

# Salida esperada tras la alineación de puertos:
{"status": "healthy"}
```

Navegación Visual en Azure Portal: En la hoja `Container Apps > ai-api > Entrada`, localice el campo **Puerto de destino**. Modifique el valor de 3000 a `8000` y presione **Guardar**. Observe en `Transmisión de registros` cómo las peticiones HTTP retornan inmediatamente con código 200 OK.

Fase 4: Observabilidad Histórica con Log Analytics y Consultas KQL

Supervisión de Registros de Consola y Sistema, Extracción de Workspace ID y Telemetría Forense

Diferenciación Arquitectónica de Logs: Los registros de consola mostrados por `az containerapp logs show` son **recientes y volátiles** (se pierden tras reinicios o recreaciones de réplicas). Para investigación forense, cumplimiento y análisis histórico, los logs persisten en el área de trabajo de **Azure Log Analytics** vinculada al Environment.

Paso 1: Extracción del Customer ID del Log Analytics Workspace

ID RESOLUTION

Obtenga el identificador único del área de trabajo asociada al Container Apps Environment para dirigir las consultas KQL:

```
# Extraer el Customer ID de Log Analytics desde las propiedades del entorno
$WORKSPACE_ID = az containerapp env show `
  -n $env:ACA_ENVIRONMENT -g $env:RESOURCE_GROUP `
  --query properties.appLogsConfiguration.logAnalyticsConfiguration.customerId `
  -o tsv

Write-Output "Workspace ID: $WORKSPACE_ID"
```

Paso 2: Últimos 20 Logs de Consola

KQL CONSOLA

Consulta sobre la tabla `ContainerAppConsoleLogs_CL`:

```
az monitor log-analytics query `
  -w $WORKSPACE_ID `
  --analytics-query "ContainerAppConsoleLogs_CL
  | where ContainerAppName_s == '$env:CONTAINER_APP_NAME'
  | project TimeGenerated, Log_s
  | order by TimeGenerated desc
  | take 20" -o table
```

Paso 3: Filtrado de Logs de Error

KQL ERRORES

Aislamiento de excepciones y fallos en runtime:

```
az monitor log-analytics query `
  -w $WORKSPACE_ID `
  --analytics-query "ContainerAppConsoleLogs_CL
  | where ContainerAppName_s == '$env:CONTAINER_APP_NAME'
    and Log_s contains 'error'
  | project TimeGenerated, Log_s
  | order by TimeGenerated desc
  | take 20" -o table
```

Paso 4: Supervisión de Eventos de Plataforma (ContainerAppSystemLogs_CL)

SYSTEM LOGS

A diferencia de los logs de aplicación, la tabla `ContainerAppSystemLogs_CL` captura eventos de la infraestructura subyacente (fallos de pull de imagen en ACR, escala a cero, fallos de sondeos y reinicios de réplica):

```
# Consultar eventos del orquestador (extracciones OCI, reinicios OOM, sondas fallidas)
az monitor log-analytics query `
  -w $WORKSPACE_ID `
  --analytics-query "ContainerAppSystemLogs_CL
  | where ContainerAppName_s == '$env:CONTAINER_APP_NAME'
  | project TimeGenerated, EventType_s, Reason_s, Log_s
  | order by TimeGenerated desc
  | take 10" -o table
```

CONSIDERACIONES OPERATIVAS Y DIRECTRICES DE OBSERVABILIDAD EN IA

- **Latencia de Ingesta (2 a 5 minutos):** Los datos de Log Analytics experimentan un retraso transitorio tras su emisión; use `az containerapp logs show` para monitoreo en vivo inmediato y Log Analytics para auditoría.
- **Trazabilidad Estructurada sin Filtración de Secretos:** Incluya siempre `request_id`, versión del modelo (`gpt-5.1`) y latencias de inferencia; **NUNCA registre contenido confidencial de prompts o documentos** para cumplir con GDPR y normativas de privacidad.

Ciclo de Vida, Sondeos de Salud, Identidad de Imagen y Optimización de Cómputo

Directrices Oficiales de Microsoft Learn para Operaciones del Día Dos en Contenedores de IA

1. IDENTIDAD DE IMAGEN: TAGS VS. DIGEST SHA-256

Slide 18 Oficial: En entornos de producción y misiones críticas, la trazabilidad del binario es prioritaria:

- **Etiquetas Mutables (`:latest`):** Son convenientes para desarrollo, pero representan un riesgo operacional al cambiar de contenido sin previo aviso.
- **Resúmenes Inmutables (`@sha256:...`):** Garantizan que todas las réplicas ejecuten *exactamente el mismo artefacto binario*, facilitando la correlación forense durante incidentes.
- **Estrategia de Retención de Revisiones:** Conserve revisiones inactivas con errores para análisis forense antes de depurarlas.

2. OPERACIONES DE CICLO DE VIDA Y MITIGACIÓN

Slide 19 Oficial: Elección de la acción adecuada según la severidad del incidente:

- **Desactivar Revisión (Recomendado):** Aísla la versión defectuosa de forma inmediata. Es el *primer paso más seguro* ante fallos de despliegue.
- **Reiniciar (Restart):** Borra estado transitorio y soluciona bloqueos de subprocesos o fugas de memoria temporales.
- **Detener (Stop):** Garantiza cero tráfico y cero consumo de cómputo. Uso reservado para incidentes mayores.

3. SONDEOS DE SALUD (PROBES) EN MODELOS DE IA

Slide 21 Oficial: Calentamiento de pesos y prevención de deadlocks:

- **Readiness Probe (Preparación):** Protege el contenedor durante la carga del modelo en memoria; configure `initialDelaySeconds` y un `failureThreshold` generoso para evitar descartes prematuros.
- **Liveness Probe (Vivacidad):** Detecta deadlocks y reinicia el pod. **NUNCA debe depender de servicios externos** (ej. Cosmos DB o APIs externas) para evitar reinicios en cascada.

4. DIMENSIONAMIENTO Y REGLAS DE ESCALADO

Slide 22 Oficial: Equilibrio entre coste, rendimiento y latencia:

- **CPU Throttling:** Si los logs reportan limitación de CPU, *aumente primero la asignación de vCPU por réplica* antes de modificar las reglas de autoescalado.
- **Reinicios OOM (Out Of Memory):** Indican que la réplica superó su cuota RAM; incremente la memoria asignada.
- **Estrategias:** APIs síncronas requieren `minReplicas ⩾ 1` para eliminar arranques en frío; workers en background escalan a cero.

Matriz de Diagnóstico y Mitigación de Errores Comunes en Tiempo de Ejecución

Patrón de Error	Síntoma Típico Observado	Causa Raíz Arquitectónica	Acción Correctiva Oficial
Image Pull Failure	Revisión en estado *Failed*, evento `ImagePullError`	Credenciales de ACR incorrectas o permiso RBAC `AcrPull` no asignado a la Managed Identity.	Configurar `az containerapp registry set` o asignar el rol RBAC `AcrPull` en el registro.
Port Mismatch	HTTP 503 Service Unavailable / Connection Refused	El Ingress reenvía tráfico a un `target-port` donde no hay ningún proceso TCP escuchando.	Alinear `target-port` con el socket de enlace de Gunicorn (`--target-port 8000`).
Missing Env Var	Valores `not-configured` o excepciones 500	Variable Twelve-Factor ausente en la plantilla del contenedor o secreto secretref no resuelto.	Inyectar la variable mediante `az containerapp update --set-env-vars` o verificar Secret Vault.
Probe Timeout	Replicas en ciclo continuo de reinicio (CrashLoop)	El modelo de IA demora más en inicializarse que el `initialDelaySeconds` del Readiness Probe.	Aumentar `initialDelaySeconds` y relajar el umbral `failureThreshold` en el probe.

Diagrama de Arquitectura Lógica de la Solución (Azure Container Apps)

Modelo Lógico de Diagnóstico, Enrutamiento de Tráfico, Gestión de Revisiones y Flujo de Telemetría

CONTEXTO EMPRESARIAL: DIAGNÓSTICO Y RESILIENCIA EN MICROSERVICIOS SERVERLESS DE IA

Escenario Empresarial: Una API de procesamiento de documentos y análisis con modelos de lenguaje (`gpt-5.1`) desplegada en Azure Container Apps experimenta fallas silenciosas de configuración y caídas perimetrales de enrutamiento tras despliegues en producción. La arquitectura implementa aislamiento mediante revisiones inmutables, conmutación granular de tráfico, verificación por sondeos de salud y observabilidad centralizada con Log Analytics.

ARQUITECTURA LÓGICA DE DIAGNÓSTICO Y FLUJO DE DATOS (DRAW.IO CANVAS VECTORIAL)

Revisiones Inmutables • Envoy Ingress • Log Analytics KQL

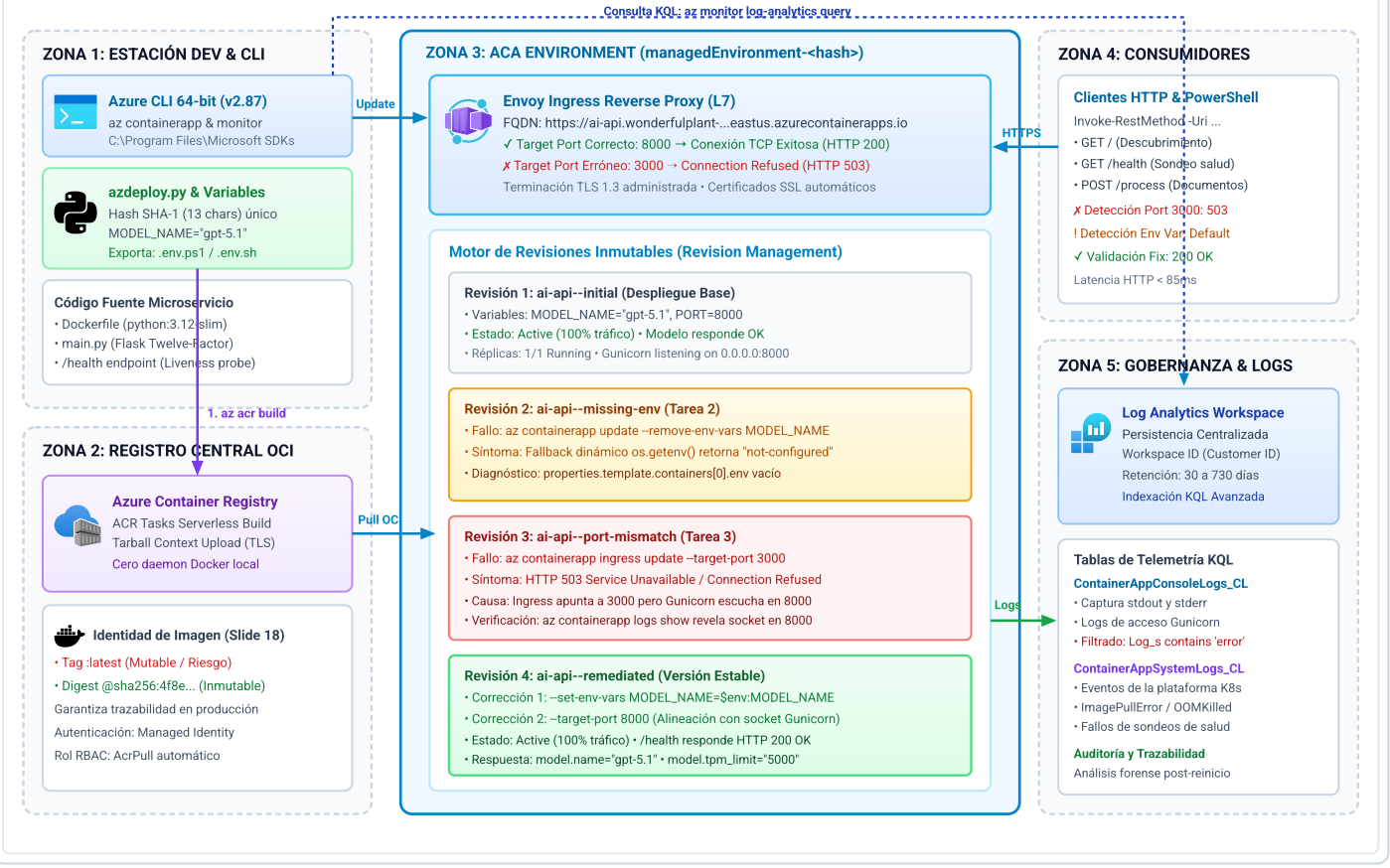


Diagrama de Arquitectura Física y Topología de Red de Azure Container Apps

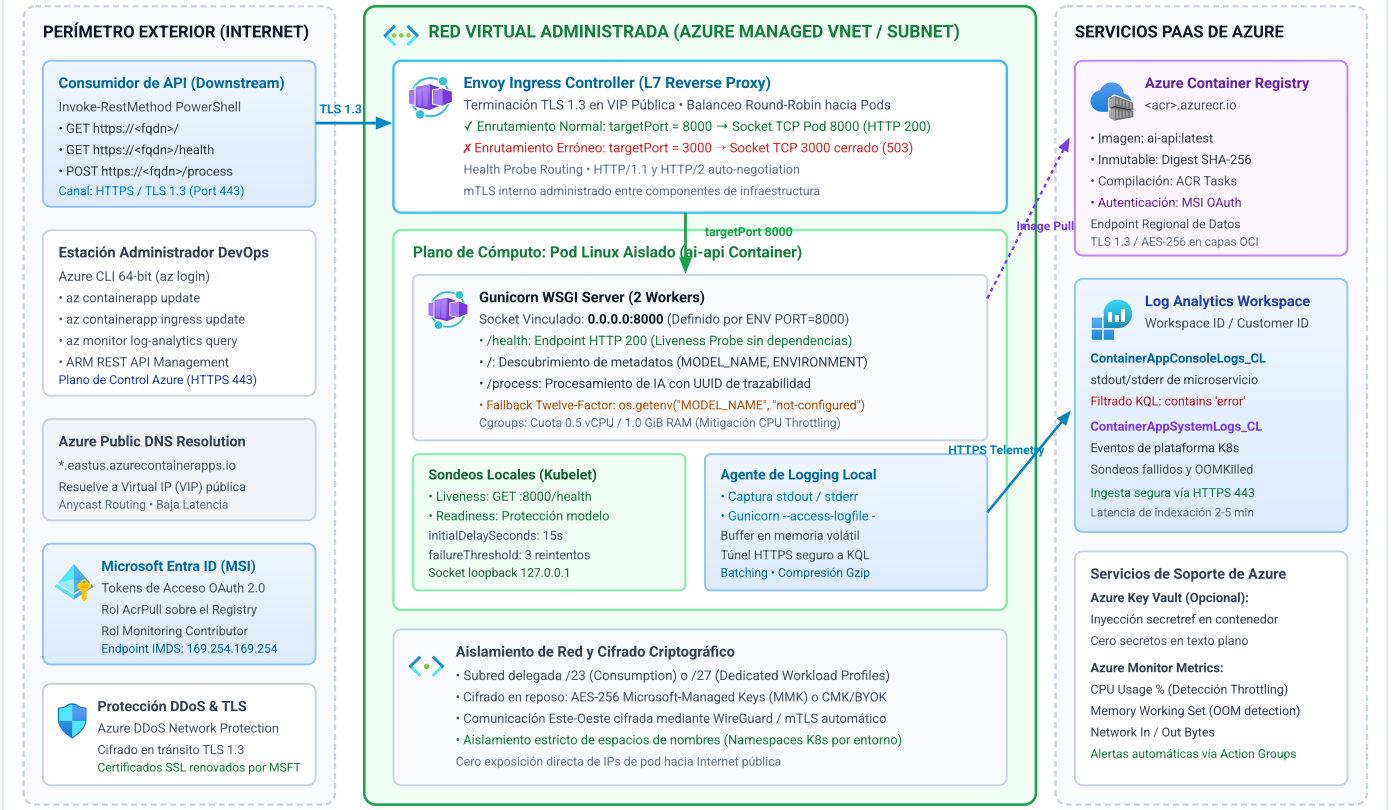
Fronteras Perimetrales, Túneles TLS 1.3, Aislamiento de Red Envoy y Flujo de Datos a Log Analytics

TOPOLOGÍA DE RED Y FRONTERAS DE AISLAMIENTO FÍSICO

Aislamiento Perimetral y Gestión de Red: Azure Container Apps encapsula las réplicas dentro de un clúster Kubernetes serverless administrado. La terminación TLS 1.3 se ejecuta en el proxy inverso Envoy perimetral. El tráfico interno hacia el socket Gunicorn (`0.0.0.0:8000`) y la telemetría enviada a Log Analytics Workspace transitan por canales privados de la red interna de Azure, garantizando confidencialidad criptográfica e integridad estructural.

TOPOLOGÍA DE RED FÍSICA Y FLUJO DE PROTOCOLOS (DRAW.IO CANVAS VECTORIAL)

TLS 1.3 • Envoy Proxy • Target Port 8000 • Ingesta KQL



Preguntas Oficiales de Examen de Certificación (AI-200 Skills Matrix)

Matriz Oficial de Distribución de Habilidades de Microsoft Learn y Casos Prácticos de Evaluación

Matriz Oficial de Dominios Evaluados — Microsoft Certified: Azure AI Cloud Developer Associate (AI-200) — STUDY GUIDE OFICIAL

Dominio 1 [★ ESTE LAB]	Dominio 2	Dominio 3	Dominio 4
Contenedores en Azure	Datos y Embeddings de IA	Consumo de Servicios Azure	Seguridad y Troubleshooting
20% – 25%	**25% – 30%**	**20% – 25%**	**20% – 25%**

Caso 1 (Slide 24 Oficial): Trazabilidad de Imagen en Despliegues de Producción

DOMINIO 1

Escenario: Implementará una nueva imagen en una aplicación contenedora de producción. ¿Qué enfoque proporciona la mejor rastreabilidad y reduce el riesgo de implementar el artefacto incorrecto?

A) Hacer referencia a la imagen por la etiqueta `latest`.
B) **[CORRECTA] Hacer referencia a la imagen por su resumen (Digest SHA-256).**
C) Recompilar la imagen localmente en cada entorno de despliegue.

> **Justificación Oficial Exhaustiva:** Las etiquetas (tags) son punteros mutables que pueden sobrescribirse involuntariamente. El resumen (Digest criptográfico SHA-256) es estrictamente inmutable; garantiza que todas las réplicas ejecuten idéntico artefacto y permite correlacionar inequívocamente el comportamiento del sistema con una compilación auditada. La opción C introduce divergencia entre entornos (environment drift).

Caso 2 (Slide 24 Oficial): Latencia y Limitación de CPU en Cargas de IA

OPTIMIZACIÓN

Escenario: Su API de IA muestra alta latencia y los mensajes de registro indican limitación de CPU (CPU throttling) durante picos de tráfico. ¿Qué cambio se dirige más directamente a resolver la limitación?

A) **[CORRECTA] Aumentar la asignación de CPU por réplica y, a continuación, volver a evaluar las reglas de escalado.**
B) Desactivar la revisión más reciente para reducir la carga de cómputo.
C) Cambiar la referencia de la imagen de resumen (digest) a etiqueta (tag).

> **Justificación Oficial Exhaustiva:** Cuando los subprocesos de inferencia de un contenedor saturan su cuota de CPU asignada por cgroups, añadir más réplicas no soluciona el problema si cada réplica individual sufre estrangulamiento. Primero debe redimensionarse la CPU por réplica para satisfacer la demanda de procesamiento y posteriormente calibrar las métricas de autoescalado. La opción B provoca indisponibilidad y la C es irrelevante.

Caso 3: Ingress 503 Mismatch

Tras actualizar `target-port` a 3000, los clientes reciben HTTP 503. ¿Cuál es la causa raíz?

Solución: Desajuste entre el proxy Envoy y el socket TCP de Gunicorn (8000). Envoy no puede reenviar tráfico a un puerto cerrado.

> **Fix:** `az containerapp ingress update --target-port 8000`.

Caso 4: Fallback Env Var

Al eliminar `MODEL_NAME`, la API responde con `not-configured` sin colapsar. ¿Por qué?

Solución: El microservicio implementa fallback con `os.getenv("MODEL_NAME", "not-configured")`, mitigando un crash fatal.

> **Principio:** Container Apps crea una nueva revisión inmutable para cada cambio.

Caso 5: Diagnóstico Forense KQL

Una réplica sufrió un reinicio OOMKilled hace 3 horas. `az containerapp logs show` no muestra registros. ¿Cómo auditarlo?

Solución: Consultar `ContainerAppConsoleLogs_CL` y `ContainerAppSystemLogs_CL` en Log Analytics Workspace con KQL.

> **Rationale:** Los logs de consola son volátiles; Log Analytics persiste hasta 730 días.

Glosario de Términos, Conceptos Clave y Siglas de Azure

Definiciones Normativas de Azure Container Apps, Diccionario de Siglas y Mapa Curricular Oficial

1. Conceptos Arquitectónicos Centrales de Azure Container Apps

REVISIÓN INMUTABLE (ACA)

Instantánea versionada que congela la imagen OCI, variables de entorno, cuotas de vCPU/RAM y secretos. Cualquier actualización crea una nueva revisión sin mutar contenedores en caliente.

TARGET PORT & ENVOY INGRESS

Puerto TCP del contenedor hacia el cual el proxy Envoy reenvía tráfico tras terminar TLS 1.3 en el FQDN. Un desajuste con el socket de Gunicorn (ej. 3000 vs 8000) genera un error 503.

IMAGE DIGEST SHA-256

Hash criptográfico inmutable que identifica unívocamente el binario de la imagen en ACR, eliminando la ambigüedad y riesgos operacionales de las etiquetas mutables (`:latest`).

READINESS PROBE EN IA

Sonda de preparación que retiene el tráfico HTTP hasta que el microservicio finaliza la carga de pesos del modelo de IA en memoria, evitando respuestas erróneas a clientes iniciales.

LIVENESS PROBE & DEADLOCKS

Sonda de vivacidad periódica que comprueba la salud interna del proceso y gatilla su reinicio ante interbloqueos; **nunca debe depender de bases de datos o servicios externos**.

LOG ANALYTICS & KQL

Repositorio centralizado persistente que almacena `ContainerAppConsoleLogs_CL` y eventos de sistema `ContainerAppSystemLogs_CL` para diagnóstico forense post-reinicio.

2. Diccionario de Siglas y Acrónimos Técnicos

Sigla	Significado y Definición Técnica
ACA	Azure Container Apps: Servicio serverless de contenedores gestionado sobre Kubernetes.
ACR	Azure Container Registry: Registro privado administrado compatible con especificación OCI.
AKS	Azure Kubernetes Service: Clúster de Kubernetes completamente administrado en la nube.
ARM	Azure Resource Manager: Capa de gestión e infraestructura declarativa de Microsoft Azure.
CLI	Command-Line Interface: Interfaz de comandos multiplataforma para gestionar Azure.
CPU	Central Processing Unit: Unidad de cómputo virtual asignada por cgroups en cada réplica.
DNS	Domain Name System: Sistema de nombres para resolución jerárquica de FQDNs públicos.
ENV	Environment Variable: Parámetros Twelve-Factor inyectados en runtime del contenedor.
FQDN	Fully Qualified Domain Name: Nombre de dominio completo asignado por Envoy Ingress.
GUI	Graphical User Interface: Consola gráfica interactiva de administración de Azure Portal.
HTTP	Hypertext Transfer Protocol: Protocolo de comunicación sin estado a nivel de aplicación.
HTTPS	Hypertext Transfer Protocol Secure: Comunicación web cifrada con certificados TLS 1.3.

Sigla	Significado y Definición Técnica
JMESPath	Lenguaje de consulta JSON empleado en Azure CLI mediante el parámetro `--query` .
KQL	Kusto Query Language: Lenguaje declarativo de consultas para Azure Log Analytics.
MSI	Managed Service Identity: Identidad administrada de Entra ID para autenticación OIDC.
OCI	Open Container Initiative: Estándar abierto para formatos de imagen y runtimes.
OOM	Out Of Memory: Estado en que el kernel de Linux mata un pod por agotar su cuota RAM.
RAM	Random Access Memory: Memoria volátil asignada a réplicas para inferencia y buffers.
RBAC	Role-Based Access Control: Control de autorizaciones basado en roles (Contributor, AcrPush).
REST	Representational State Transfer: Arquitectura de servicios web mediante verbos HTTP.
SKU	Stock Keeping Unit: Nivel comercial de servicio en Azure (Basic, Standard, Premium).
TLS	Transport Layer Security: Protocolo criptográfico perimetral para cifrado en tránsito.
TPM	Tokens Per Minute: Límite de tasa de tokens admitidos por despliegues de modelos de IA.
VNet	Virtual Network: Red virtual de Azure para aislamiento y segmentación de subredes.

3. Mapa Curricular y Rutas de Aprendizaje Oficiales del Curso AI-200T00-A

FASE 1: CÓMPUTO & CONTENEDORES

Dominio 1: Contenedores (20% – 25%)

- Módulo 01: Azure CLI & ACR Tasks
- Módulo 02: Azure App Service Containers
- Módulo 03: Despliegue en Container Apps
- **Módulo 04: Diagnóstico en ACA [★ Este Lab]**
- Módulo 05: Azure Kubernetes Service (AKS)

FASE 2: DATOS & EMBEDDINGS

Dominio 2: Datos de IA (25% – 30%)

- Módulo 06: Cosmos DB Vector Search
- Módulo 07: Azure Database for PostgreSQL
- Módulo 08: Azure Managed Redis Semantic
- Módulo 09: Azure AI Search Integration

FASE 3: INTEGRACIÓN & DEVOPS

Dominios 3 y 4: Conexión & Seguridad

- Módulo 10: Azure Service Bus & Event Grid
- Módulo 11: Azure Key Vault & Managed ID
- Módulo 12: Azure Monitor & OpenTelemetry
- Módulo 13: CI/CD Pipelines con GitHub Actions

Recurso Oficial Microsoft Learn: Guía de Estudio de Certificación AI-200 disponible en https://learn.microsoft.com/es-es/credentials/certifications/resources/study-guides/ai-200